✅ Practical 1: Recursion

Q1. What is recursion? Why is base case important?
Recursion is a method where a function calls itself to solve smaller parts of a problem.
The base case is important because it stops the recursion; without it, the function would run infinitely.

Q2. How many rods are used in Tower of Hanoi?
Three rods are used: Source, Auxiliary, and Destination.
Disks are moved from source to destination using the auxiliary rod.

Q3. Compare recursion and iteration.
Recursion uses function calls and stack memory, while iteration uses loops.
Recursion is easier to write but uses more memory than iteration.

Q4. State advantages and disadvantages of recursion.
Recursion makes code simple and easy for complex problems.
However, it uses more memory and may be slower due to repeated function calls.

Q5. Where is recursion used in real life?
It is used in tree structures, file systems, and algorithms like divide-and-conquer.
Examples include searching folders and solving puzzles.

✅ Practical 2: Sorting Algorithms

Q1. Define sorting. Why is it important?
Sorting means arranging data in a specific order like ascending or descending.
It helps in faster searching and better organization of data.

Q2. What is adaptive sorting? Which algorithm is adaptive?
Adaptive sorting takes advantage of already sorted data to reduce work.
Insertion Sort is adaptive because it works faster for nearly sorted lists.

Q3. Compare Bubble, Insertion, Selection Sort.
Bubble Sort repeatedly swaps elements, Insertion inserts elements in correct position, Selection selects minimum element.
Insertion is faster for small data, while Selection does fewer swaps.

Q4. Which sorting algorithm is best for nearly sorted list and why?
Insertion Sort is best because it requires fewer comparisons and shifts.
It works efficiently when data is already partially sorted.

Q5. Give real-life applications of each algorithm.
Sorting is used in ranking systems, organizing records, and searching.
Insertion Sort is like arranging cards, while Selection is used in simple systems.

✅ Practical 3: Binary Search & Sorting

Q1. Why must array be sorted for Binary Search? What is its time complexity?
Binary Search works by dividing data into halves, which requires sorted data.
Its time complexity is O(log n), making it very efficient.

Q2. Can Binary Search be applied to linked lists? Why?
No, because linked lists do not allow direct access to middle elements.
Binary Search needs indexing, which linked lists lack.

Q3. Why is Merge Sort called stable?
Merge Sort keeps the relative order of equal elements unchanged.
This makes it stable and useful for certain applications.

Q4. Why Quick Sort has worst case $O(n^2)$?
If the pivot is always the smallest or largest element, partitions become unbalanced.
This leads to poor performance.

Q5. Compare Merge Sort and Quick Sort.
Merge Sort is stable and uses extra memory, while Quick Sort is faster and in-place.
Quick Sort performs better on average.

✅ Practical 4: Greedy Method

Q1. Why greedy method not always optimal?
Greedy makes the best local choice at each step.
But sometimes local choices do not lead to the best overall solution.

Q2. What is greedy choice property?
It means choosing the best option at each step leads to optimal solution.
This property is required for greedy algorithms to work correctly.

Q3. What is optimal substructure?
It means a problem can be solved using solutions of smaller subproblems.
Many optimization problems follow this property.

Q4. Why sorting is required in greedy algorithms?
Sorting helps in selecting the best elements first based on criteria.
For example, selecting highest profit or earliest finish time.

Q5. Give real-life applications of greedy algorithms.
Used in scheduling, network routing, and resource allocation.
Example: choosing shortest path in maps.

✅ Practical 5: Dynamic Programming

Q1. What is Dynamic Programming? Why is it preferred?
DP solves problems by storing results of subproblems to avoid repetition.
It is preferred because it improves efficiency over brute force.

Q2. Difference between Fractional and 0/1 Knapsack.
Fractional allows breaking items, while 0/1 does not.
0/1 uses DP, while fractional uses greedy approach.

Q3. Objective of Coin Change Problem?
To find minimum number of coins needed for a given amount.
If no combination exists, the problem has no solution.

Q4. Difference between Job Sequencing and Weighted Job Scheduling.
Job Sequencing focuses on deadlines, while Weighted Scheduling considers time intervals and profit.
Weighted version is more complex.

Q5. Real-life applications of DP.
Used in finance, resource planning, and shortest path problems.
Example: route optimization in GPS.

✅ Practical 6: Graph Algorithms

Q1. What is a graph? Explain components.
A graph consists of vertices (nodes) and edges (connections).
It represents relationships between objects.

Q2. Difference between single-source and all-pairs shortest path.
Single-source finds shortest path from one node to all others.
All-pairs finds shortest paths between every pair of nodes.

Q3. What is negative edge weight?
An edge with a negative cost or value.
It can reduce total path cost.

Q4. Why is algorithm analysis important?
It helps in choosing the most efficient algorithm.
This saves time and resources.

Q5. Applications of Bellman-Ford, Floyd-Warshall, TSP.
Used in routing, traffic systems, and delivery optimization.
TSP helps in finding shortest tour.

✅ Practical 7: Backtracking

Q1. Difference between brute force and backtracking.
Brute force checks all possibilities, while backtracking skips invalid ones.
This makes backtracking more efficient.

Q2. What is a state space tree?
It is a tree representing all possible solutions of a problem.
Each node represents a partial solution.

Q3. Why Subset Sum is NP-Complete?
Because it has no known efficient solution for large inputs.
All combinations must be checked.

Q4. Constraints in 8-Queens Problem.
No two queens can be in the same row, column, or diagonal.
These constraints ensure a valid solution.

Q5. Real-life applications.
Used in puzzles, AI problems, and decision-making systems.
Example: Sudoku solving.

✅ Practical 8: Backtracking & Graph

Q1. Why backtracking better than brute force?
It avoids exploring invalid solutions early.
This reduces time complexity.

Q2. How many solutions exist for 8-Queens problem?
There are 92 possible solutions.
This shows complexity of the problem.

Q3. Why graph coloring NP-Complete?
Because it requires checking many color combinations.
No fast algorithm exists for all cases.

Q4. Why Hamiltonian Cycle NP-Complete?
It requires checking all possible paths in a graph.
This makes it computationally expensive.

Q5. Real-life applications.
Used in scheduling, map coloring, and network design.
Helps avoid conflicts.

✅ Practical 9: Stack & Queue

Q1. What is node selection strategy?
It is the method of choosing which node to process next.
It affects how algorithms explore data.

Q2. Difference between LIFO and FIFO.
LIFO uses stack (last element first), FIFO uses queue (first element first).
They define different processing orders.

Q3. Role in Branch and Bound.
Node selection decides search direction and efficiency.
It helps in exploring solution space.

Q4. Which strategy requires more memory and why?
FIFO requires more memory because it stores more nodes at once.
LIFO uses less memory.

Q5. Applications of stack and queue.
Stack: recursion, undo operations
Queue: scheduling, BFS traversal

✅ Practical 10: Branch & Bound

Q1. What is Branch and Bound?
It is an optimization technique that divides problems into smaller parts.
It eliminates non-promising solutions using bounds.

Q2. Difference between FIFO and LC Branch & Bound.
FIFO uses queue and explores level-wise.
LC uses priority queue and selects best node first.

Q3. What is a bounding function?
It estimates the best possible solution from a node.
It helps in pruning unnecessary paths.

Q4. Which problems are suitable for Branch and Bound?
Knapsack, Travelling Salesman Problem, scheduling.
These involve optimization.

Q5. Why LC is more efficient than FIFO?
LC selects the most promising node first.
This reduces unnecessary exploration and improves performance.

EXTRA

✅ Practical 1: Recursion (Extra)

Q1. What is stack overflow in recursion?
It happens when too many recursive calls fill the stack memory.
This usually occurs if the base case is missing or incorrect.

Q2. What is tail recursion?
Tail recursion is when the recursive call is the last statement in the function.
It can be optimized by the compiler to improve performance.

Q3. What is indirect recursion?
Indirect recursion occurs when one function calls another, which again calls the first.
Example: A → B → A.

Q4. How to convert recursion to iteration?
By using loops and a stack data structure.
This avoids function call overhead.

✅ Practical 2: Sorting (Extra)

Q1. What is stability in sorting?
A stable sort keeps the order of equal elements unchanged.
Example: Insertion Sort is stable.

Q2. What is in-place sorting?
Sorting that uses constant extra memory.
Example: Selection Sort, Bubble Sort.

Q3. What is time complexity?
It measures how execution time grows with input size.
Used to compare algorithms.

Q4. What is best case in sorting?
It is the minimum time taken when input is already sorted.
Example: O(n) for Insertion Sort.

✅ Practical 3: Searching & Sorting (Extra)

Q1. What is divide and conquer?
A technique that divides a problem into smaller parts, solves them, and combines results.
Used in Merge Sort and Quick Sort.

Q2. What is pivot in Quick Sort?
Pivot is the element used to divide the array.
Elements are arranged around it.

Q3. What is linear search?
It checks each element one by one.
Time complexity is O(n).

Q4. Why is Binary Search faster?
Because it reduces the search space by half each time.
Hence it has O(log n) complexity.

✅ Practical 4: Greedy (Extra)

Q1. What is greedy strategy?
It selects the best option at each step.
It does not reconsider previous choices.

Q2. What is local optimum?
Best solution at a particular step.
May not be globally best.

Q3. What is global optimum?
Best possible solution overall.
Goal of optimization problems.

Q4. Give example where greedy fails.
0/1 Knapsack problem.
Greedy may not give optimal result.

✅ Practical 5: Dynamic Programming (Extra)

Q1. What are overlapping subproblems?
Same subproblems are solved multiple times.
DP stores results to avoid repetition.

Q2. What is memoization?
Storing results of recursive calls.
Used to improve efficiency.

Q3. What is tabulation?
Bottom-up DP approach using tables.
Starts from smallest subproblems.

Q4. Difference between DP and recursion?
DP stores results; recursion may recompute.
DP is more efficient.

✅ Practical 6: Graph Algorithms (Extra)

Q1. What is a weighted graph?
A graph where edges have values (weights).
Used in shortest path problems.

Q2. What is a cycle in graph?
A path that starts and ends at same vertex.
Important in graph analysis.

Q3. What is adjacency matrix?
A 2D matrix representing edges between vertices.
Used to store graph.

Q4. What is a path?
Sequence of vertices connected by edges.
Used in traversal.

✅ Practical 7: Backtracking (Extra)

Q1. What is pruning?
Removing unwanted solutions early.
Improves efficiency.

Q2. What is feasibility check?
Checking if a solution satisfies constraints.
Used before moving forward.

Q3. What is recursion tree?
Tree showing recursive calls.
Helps visualize execution.

Q4. Why backtracking is recursive?
Because it explores solutions step-by-step using recursion.
Returns when solution fails.

✅ Practical 8: Backtracking & Graph (Extra)

Q1. What is constraint satisfaction problem?
Problem where solution must satisfy given constraints.
Example: N-Queens.

Q2. What is graph representation?
Ways to store graph (matrix or list).
Helps in algorithm implementation.

Q3. What is vertex degree?
Number of edges connected to a vertex.
Important property of graph.

Q4. What is simple graph?
Graph without loops or multiple edges.
Basic graph type.

✅ Practical 9: Stack & Queue (Extra)

Q1. What is stack overflow?
Occurs when stack is full.
Cannot push more elements.

Q2. What is queue overflow?
Occurs when queue is full.
Cannot insert more elements.

Q3. What is deque?
Double-ended queue.
Insertion/deletion at both ends.

Q4. What is priority queue?
Queue where elements have priority.
Highest priority is served first.

✅ Practical 10: Branch & Bound (Extra)

Q1. What is branching?
Dividing problem into smaller subproblems.
Creates a solution tree.

Q2. What is pruning?
Removing non-promising nodes.
Reduces computation.

Q3. What is state space tree?
Tree representing all possible solutions.
Used in optimization.

Q4. Difference between backtracking and Branch & Bound?
Backtracking checks feasibility.
Branch & Bound uses bounds to optimize.